

METRICAS

1. MLP

2. CONTROL/LOCOMOCIÓN

A) Tasa de éxito

Se considera una prueba exitosa si el robot completa el recorrido sin vuelco, sin intervención manual y manteniendo el desplazamiento.

$$SuccessRate = \frac{N_{success}}{N_{total}}$$

Tabla:

Escenario	Pruebas	Éxitos	Tasa	Std
Plano	N		%	
Rampa	N		%	
Terreno irregular	N		%	

B) Tiempo de respuesta

- Cambio de terreno
- estabilización del nuevo estado locomotor

$$T_{response} = t_{stable} - t_{change}$$

tabla:

Escenario	Tiempo medio (s)	Std
Plano→rampa		
Plano→irregular		

C) Delay de cambio de modo

Tiempo que emplea PETER al cambiar de un modo a otro

$$T_{switch} = t_{stabilized} - t_{cmd}$$

Procedimiento:

1. **tcmd**: Registra el tiempo exacto en el milisegundo en que tu red neuronal (Ganglios Basales) cambia la variable de estado o usando un umbral condicional
2. **tstabilized**: Define un criterio de estabilización mecánica. Por ejemplo, cuando los encoders de las patas alcanzan sus posiciones de caminata y la velocidad angular del IMU (wz, wx, wy) cae por debajo de un umbral de tolerancia (indicando que el robot dejó de tambalearse tras el cambio).

Escenario	Tiempo medio (s)	Std.
Móvil→Cuadrúpedo		
Móvil→Omnidireccional		
Cuadrúpedo → Móvil		

D) Repetibilidad durante las transiciones

Usando la media y desviación estándar del delay de cambio de modo, tiempo de respuesta y tasa de éxito podemos obtener su repetibilidad

$$Repeatability = 1 - \frac{\sigma}{\mu}$$

E) Tasa de Adaptabilidad al terreno

Es la proporción de terrenos de diferente dificultad que el robot puede superar de manera autónoma sin intervenciones o caídas.

Terrain Adaptability Rate

$$TAR = \frac{N_{exitosos}}{N_{totales}}$$

Donde Ntest es el conjunto de entornos de prueba distintos (diferentes pendientes, rugosidades, obstáculos).

F) Tracking accuracy (Precisión de seguimiento)

Error entre trayectoria deseada y real:

$$RMSE = \sqrt{\frac{1}{N} \sum (x_i - x_{ref})^2}$$

Registrar:

- error longitudinal
- error lateral

3. Métricas de estabilidad física

A) Margen de estabilidad (Tarea de SAM)

Aproximación simple usando IMU (recomendado)

Con IMU y altura aproximada del CoM:

$$ZMP_x = x_{CoM} - \frac{h}{g} \ddot{x}$$
$$ZMP_y = y_{CoM} - \frac{h}{g} \ddot{y}$$

donde:

- h : altura del CoM
- g : gravedad
- $\ddot{x}, \ddot{y}$: aceleraciones horizontales

Código

```

import numpy as np

g = 9.81
h = 0.18    # altura CoM aprox (m)

x_com = 0
y_com = 0

ax = 0.6    # m/s²
ay = -0.3

zmp_x = x_com - (h/g)*ax
zmp_y = y_com - (h/g)*ay

print(zmp_x, zmp_y)

```

Margen de estabilidad

Lo importante realmente no es el ZMP exacto, sino si cae dentro del polígono de soporte.

Entonces defines:

$$SM = d(ZMP, SupportPolygonEdge)$$

- positivo → estable
- negativo → riesgo de vuelco

B) Riesgo de vuelco

PENDIENTE

$$TiltIndex = \sqrt{Roll^2 + Pitch^2}$$

C) Amplitud de oscilación

RMS:

$$RMS_{\theta} = \sqrt{\frac{1}{N} \sum \theta_i^2}$$

Registrar:

- **Roll RMS**
- **Pitch RMS**

```
pitch_values = []
roll_values = []

# durante ejecución
pitch_values.append(pitch)
roll_values.append(roll)

# al terminar experimento
rms_pitch = np.sqrt(np.mean(np.square(pitch_values)))
rms_roll = np.sqrt(np.mean(np.square(roll_values)))

print("Pitch RMS:", rms_pitch)
print("Roll RMS:", rms_roll)
```

D) Consumo energético

Procedimiento:

1. Medición eléctrica

Mide el voltaje de la batería:

$$V(t)$$

y la corriente total consumida:

$$I(t)$$

en cada instante t .

2. Potencia instantánea

$$P(t) = V(t) \cdot I(t)$$

3. Energía total

Integra la potencia a lo largo del tiempo del experimento para obtener la energía total en Julios:

$$E = \sum(P(t) \cdot \Delta t)$$

4. Distancia recorrida

Mide la distancia neta recorrida:

$$d$$

usando la odometría del robot.

5. Cálculo del CoT

$$CoT = \frac{E}{m \cdot g \cdot d}$$

Nota: Si demuestras que el CoT en modo ruedas es drásticamente menor que en modo patas para terreno plano, habrás justificado perfectamente ante el Revisor #2 la necesidad de tu robot híbrido.

4. MÉTRICAS NEURONALES/ESTABILIDAD

Métricas neuronales (ganglios basales)

A) Latencia de decisión

Piden:

decision latency

$$T_{latency} = t_{decision} - t_{stimulus}$$

B) Estabilidad de disparo

Piden:

firing stability

$$\sigma^2 = \frac{1}{N} \sum (x_i - \mu)^2$$

C) Consistencia temporal

Nueva métrica:

temporal consistency

Correlación entre ejecuciones:

$$Corr = corr(x_1, x_2)$$

D) Eficiencia de resolución de conflictos

Nueva:

conflict-resolution efficiency

Podría definirse:

$$CRE = \frac{N_{correct}}{N_{conflicts}}$$

E) Verificación winner-take-all

Nueva.

$$WTA = \frac{\max(response)}{\sum response}$$

Valores próximos a:

1

indican dominancia clara.

5. Métricas de robustez

A) Sensibilidad a ruido

Piden:

sensor noise

Añadir ruido artificial:

- IMU $\pm 5\%$
- cámara
- LiDAR

Medir degradación:

$$Degradation = \frac{Metric_{noise}}{Metric_{normal}}$$

B) Sensibilidad paramétrica

Piden:

robustness verification

Variar:

tau
bias
threshold

Registrar:

- **tasa éxito**
- **delay**

C) Análisis de convergencia

Para sistemas de control continuos o algoritmos dinámicos recurrentes (como las dinámicas de membrana celular o redes neuronales continuas), se utiliza el **Criterio de Estabilidad de Lyapunov**. Se propone una función escalar candidata de energía $V(\mathbf{x})$ (donde $\mathbf{x}$ es el estado del sistema).

- **Condiciones matemáticas:**

1.

$$V(\mathbf{x}) > 0 \forall \mathbf{x} \neq 0 \text{ y } V(0) = 0$$

(Definida positiva)

2.

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} \dot{\mathbf{x}} \leq 0$$

(Derivada semi-definida negativa para estabilidad en el sentido de Lyapunov)

3.

$$\dot{V}(\mathbf{x}) < 0$$

(Definida negativa para **convergencia asintótica**, lo que garantiza que los estados llegarán al equilibrio deseado)

6. TERRENO (COMPLETO)

A) Media altura

$$\mu_h = \frac{1}{N} \sum h_i$$

B) Varianza

$$\sigma_h^2 = \frac{1}{N} \sum (h_i - \mu)^2$$

C) Rango

$$Range = h_{max} - h_{min}$$

D) Pendiente media

$$Slope = \tan^{-1} \left(\frac{\Delta h}{\Delta x} \right)$$